Synopsis

Movie Preference Prediction System using Machine Learning (Netflix Dataset)

Problem Id: 17

Student Name : Vanshika Ahuja(Team Leader)

University Roll number : 2028900

Section : K G2

Student Name : Priyanka Rawat

University Roll number : 2028807

Section : K G2

Problem Description:

- Online streaming platforms such as Netflix provide a large collection of movies and shows, making it difficult for users to choose what to watch.
- Recommendation systems help users discover movies by analyzing their past viewing and rating behavior.
- The goal of this project is to build a machine learning system that predicts whether a user will like a movie based on previous ratings.
- The dataset contains **User ID, Movie ID, Rating, and Timestamp**, which represent the user's interaction with movies.
- Data preprocessing will include handling missing values, removing duplicate records, and encoding categorical attributes such as user IDs and movie IDs.
- Ratings will be converted into two categories:
- **1 → High Rating / Recommended**
- **0 → Low Rating / Not Recommended**
- Machine learning models such as **Logistic Regression, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)** will be trained to classify user preferences.
- The models will be evaluated using metrics like **accuracy, precision, recall, F1-score, confusion matrix, and ROC curve** to determine the best-performing model for movie recommendation.

Explanation of Each Phase of the ML Pipeline:

- ### 1. Dataset Collection

- The first step is collecting the dataset required for the project. In this project, the movie rating dataset from MovieLens is used. The dataset contains information such as **userId, movieId, rating, and timestamp**, which represent user interactions with movies.

- ### 2. Data Preprocessing

- Raw datasets usually contain noise or inconsistencies. In this stage:
- Missing values are handled or removed.

- Duplicate records are eliminated.
- Ratings are converted into binary values where **rating ≥ 3 = 1 (Recommended)** and **rating < 3 = 0 (Not Recommended)**.
- Categorical features such as **userId and movieId** are encoded so that machine learning models can process them.

3. Feature Engineering

- Important features are created from the existing data to improve model performance. Examples include:
- **User average rating**
- **Movie popularity (number of ratings)**
- **Historical rating patterns**
- These features help the model understand user behavior and movie trends.

4. Train–Test Split

- The dataset is divided into two parts:
- **Training set (80%)** – used to train the machine learning models.
- **Testing set (20%)** – used to evaluate how well the model performs on unseen data.

5. Model Training

- Three classification algorithms are used:
- **Logistic Regression**
- **K-Nearest Neighbors (KNN)**
- **Support Vector Machine (SVM)**
- These models learn patterns from the training data to predict user preferences.

6. Model Prediction

- After training, the models predict whether a user will give a **high rating or low rating** to a movie.

- ## 7. Model Evaluation

- The performance of the models is measured using:
- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **Confusion Matrix**
- **ROC Curve**
- These metrics help determine which model performs best.

- ## 8. Final Output

- The trained system produces a prediction:
- **1 → High Rating / Recommended**
- **0 → Low Rating / Not Recommended**

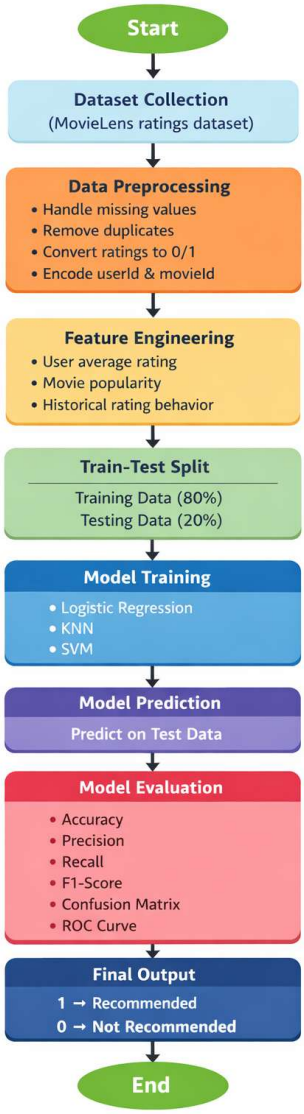

Fig. 1 Sample architecture

Expected Modules/ Libraries to be used :

- **Pandas** – Used for loading, cleaning, and manipulating the dataset using DataFrame structures.
- **NumPy** – Used for numerical operations and creating conditional labels such as Like/Dislike using `np.where()`.
- **Scikit-learn** – Provides tools for preprocessing data and building machine learning models.

- **MinMaxScaler** – Used to normalize numerical values into a range of 0–1 for better model performance.
- **Matplotlib** – Used for visualizing results such as graphs and ROC curves.

Expected Evaluation metrics and Plots to be used:

To evaluate the performance of the machine learning models, several evaluation metrics will be used.

1. **Accuracy** – Measures the overall correctness of the model by calculating the percentage of correctly predicted instances.
2. **Precision** – Indicates how many of the predicted positive cases are actually correct. It is useful when false positives need to be minimized.
3. **Recall** – Measures the ability of the model to correctly identify all positive instances. It helps in understanding how well the model captures relevant cases.
4. **F1-Score** – The harmonic mean of precision and recall, providing a balanced measure when both false positives and false negatives are important.
5. **Confusion Matrix** – A table used to visualize the performance of a classification model by showing correct and incorrect predictions.

Visualization Plots

- **Confusion Matrix Plot** – Used to visually analyze the number of correct and incorrect predictions made by the model.
- **ROC Curve (Receiver Operating Characteristic Curve)** – Used to evaluate the classification performance by plotting the true positive rate against the false positive rate. It helps compare different models effectively.

These metrics and plots help in understanding the accuracy, reliability, and overall performance of the machine learning models used in the project.

Required topics from the subject:

- **Variables** – Variables are named memory locations used to store values that can be used and changed during program execution.

- **Data Types** – Data types define the type of data a variable can store, such as integer, character, or floating-point values.

- **Operators** – Operators are symbols used to perform arithmetic, logical, or relational operations on variables and values.

- **Control Statements** – Control statements manage the flow of program execution using conditions and loops.

- **Arrays** – Arrays are collections of elements of the same data type stored in contiguous memory locations.

- **Functions** – Functions are blocks of code designed to perform a specific task and improve code reusability.

- **Strings** – Strings are sequences of characters stored in a character array and terminated by a null character (`\0`).

- **Pointers** – Pointers are variables that store the memory address of another variable.

- **Structures** – Structures are user-defined data types that group different types of variables under a single name.

- **File Handling** – File handling allows programs to read, write, and store data permanently in files.

- **Dynamic Memory Allocation** – Dynamic memory allocation allows memory to be allocated and freed during program execution.

- **Recursion** – Recursion is a programming technique in which a function calls itself to solve smaller instances of a problem.

Platform Required:

- **Jupyter Notebook** – Used to write and run machine learning code step-by-step, visualize results, and explain the workflow in the same file.

Books and Link Sources:

- Mark E.Fenner– Machine learning with python for everyone
- Tom Mitchell – Machine Learning
- https://scikit-learn.org
- https://www.kaggle.com/datasets